

Teoria 8 - Timelines

1 — TIMELINES (concepto y propósito)

- **Pregunta clave:** Una pregunta comúnmente hecha a los investigadores forenses es "*¿Qué pasó?*".
 - **Propósito:** La respuesta suele tomar la forma de una **historia** que transmite la comprensión de mecanismos e interacciones complejas de forma sencilla y directa.
 - **Organización cronológica:**
 - Una de las maneras más simples e intuitivas de organizar esa historia es **cronológicamente** (por fechas/timestamps).
 - **Ventaja:** simplifica la narración y ayuda a correlacionar eventos.
 - **Limitación práctica:**
 - Aunque la organización cronológica es simple e intuitiva, **pocas herramientas forenses** apoyan una visión útil de la misma en forma visual o navegable.
-

2 — ¿Qué es una TIMELINE?

- **Definición práctica (texto original):**
 - *Una timeline es un listado de todos los archivos del sistema ordenados por fecha.*
 - **Función:** Nos permite tener una visión general del **acceso, modificación y creación** de los archivos.
- **Utilidad:**
 - Con una timeline podemos ver si se han **borrado, modificado o accedido** archivos importantes.
 - Detectar **saltos de tiempo** (p.ej. dispositivo apagado, cambio de fecha/timezone).
 - Reconocer **secuencias conocidas**: inicio, actualizaciones, logins del sistema, conexiones SSH, etc.

3 — CREACIÓN DE UNA TIMELINE (herramientas clásicas: fls + mactime)

- **Método popular:** usar **mac-time** (herramientas del The Sleuth Kit).
- **Comandos clave:**
 - **fls** : toma como entrada una imagen y retorna una **lista de todos los archivos**. (Formato **.body** en los ejemplos.)
 - **mactime** : toma la lista producida por **fls** , analiza todos los archivos y devuelve una **línea de tiempo en formato ASCII** o CSV.

- **Ejemplo (NTFS) — fls:**

- Comando de ejemplo:

```
fls -z GMT -s 0 -m C: -f ntfs -r disco-c.dd > hddisk-c.body
```

- Salida tipo (fragmento):

```
0|C:/WINDOWS/inf/mdmpin.PNF|4718-128-3|r/rrwxrwxrwx|0|0|192
68|1299255392|1299255392|1299255392|129925
[...]
```

- **Explicación:** **fls** extrae entradas de archivos/directorios con metadatos (incluye timestamps).

- **Ejemplo — mactime:**

- Comando de ejemplo:

```
mactime -b hddisk-c.body -Z GMT -D > disk-c.csv
```

- Salida tipo (fragmento):

```
Fri Mar 04 2011 16:08:04 618605 .ac. r/rrwxrwxrwx C:/WINDOWS/s
ystem32/dllcache/fp4autl.dll 0 0 10618-128-1
```

[...]

- **Explicación:** `mactime` convierte las listas de `fls` a una timeline legible por humanos (fechas, acciones, paths).

4 — MACTIME Y LOS FILESYSTEM (nota)

- **Tener en cuenta:** el comportamiento, los mactimes y la interpretación de timestamps depende del **filesystem** (NTFS, ext, HFS+, etc.). Es necesario conocer particularidades del sistema bajo análisis para interpretar correctamente la timeline.

5 — IMPORTANCIA DE DOCUMENTACIÓN

- **Referencia:** "File System Analysis" (menciona que la documentación del filesystem es importante).
- **Consejo práctico:** documenta comandos, parámetros y versiones de herramientas usadas en la fase de examinación.

6 — TIMELINES y la investigación (paralelo con detective de TV)

- Preguntas cronológicas habituales: "¿Dónde estaba usted en el momento del incidente?", "¿hora estimada de muerte?", "¿cuando fue la última llamada de su móvil?", "¿cuál es su coartada?".
- **Analogía:** Los procesos que siguen los detectives (reconocimiento, consideración, eliminación) se realizan en el mismo orden y por las mismas razones que un investigador forense digital.

7 — Correlación de evidencias

- A medida que avanza la investigación, la **cantidad de pruebas** que deben correlacionarse aumenta.

- Un dato aparentemente menor puede convertirse en **evidencia masiva** cuando se coloca junto con otros datos (especialmente si la **ubicación** y **timestamps** son similares).
-

8 — VISUALIZACIÓN DE TIMELINES

- **Valor de la visualización:** expertos dependen fuertemente de la visualización para acelerar decisiones y análisis.
 - **Beneficios en forensia digital:** visualizar múltiples líneas de tiempo ayuda a:
 - Resúmenes ejecutivos para clientes.
 - Agilizar el trabajo del investigador.
 - Reducir tiempos de decisión.
 - **Ejemplo ilustrativo en el texto:** una timeline parcial de personajes de *Rick and Morty* por temporada/episodio — útil para acotar episodios relevantes cuando se busca un evento con personajes concretos.
-

9 — PROBLEMAS con timelines grandes

- Las timelines pueden ser **muy difíciles de estudiar** cuando el sistema analizado es muy grande.
 - **Solución:** usar visualizadores de timelines (por ejemplo **Timesketch**, **Kibana**) que permiten filtrar, visualizar y navegar grandes cantidades de eventos.
-

10 — TIMELINES: resumen práctico

- **Definición reiterada:** listado de todos los archivos ordenados por fecha — visión general de acceso/modif/creación.
 - **Fase de examinación:** una de las herramientas más útiles.
 - **Casos de uso:** rastrear sospechas, verificar hechos, detectar apagados, cambios de timezone, secuencias típicas.
-

11 — CREACIÓN DE TIMELINES (repetición / énfasis)

- Reiteración de que **fls** + **mactime** son un método clásico.
 - Incluir ejemplos y comandos (ya presentados).
-

12 — HERRAMIENTA: AUTOPSY (UI para The Sleuth Kit)

- **Autopsy**: plataforma de forensia digital que ofrece interfaz gráfica para varias aplicaciones forenses (incluyendo The Sleuth Kit).
 - **Uso común**: ampliamente utilizada para análisis forense; versiones principales: **Autopsy 2** y **Autopsy 4**.
 - **Mejora**: Autopsy4 mejora el análisis de timelines con interfaz gráfica y navegación.
-

13 — SUPER TIMELINE / enriquecimiento con otras fuentes

- **Idea**: agregar fuentes adicionales a una timeline para enriquecer contenido (no limitarse solo a timestamps de archivos).
 - Ejemplos: contenido de autenticación del sistema (`auth.log`), logs del servidor web.
 - **Valor práctico**: ver, por ejemplo, un acceso por SSH y luego los archivos que fueron modificados — permite correlacionar acceso y acciones sobre sistema.
-

14 — LOG2TIMELINE (versión antigua y evolución)

- **log2timeline** (vieja, perl versión):

- Herramienta que parsea recursivamente archivos de una partición montada.
 - Inserta datos de diferentes fuentes en la timeline (Apache2 logs, Chrome history, IE history, Squid access, etc.).
 - Puede exportarse a varios formatos.
 - **Evolución → Plaso:**
 - Se creó **Plaso** para reemplazar la versión vieja de `log2timeline`. Es ahora una aplicación completa con módulos y funciones para generación/análisis de timelines.
-

15 — PLASO (Plaso) — descripción general

- **¿Qué es Plaso?** Herramienta forense para la generación y análisis de timelines; framework que soporta:
 - agregar parsers,
 - agregar plugins de análisis,
 - automatizar tareas repetitivas,
 - agregar tags a eventos particulares.
 - **Repositorio:** <https://github.com/log2timeline/plaso>
-

16 — PLASO: ETAPAS (4 etapas)

Cuando se usa Plaso se consideran cuatro etapas **claras**:

1. Preprocesamiento (Preprocessing)

- Extrae datos de la imagen que serán utilizados en etapas siguientes: zona horaria, enumera usuarios y paths, hostname, aplicaciones por defecto (navegadores), elementos específicos del SO (carga parsers según SO).

2. Recolección (Collection)

- Lee la imagen en estudio buscando archivos que puedan ser analizados más tarde (síntesis: recolecta archivos relevantes).

3. Extracción (Extraction / Worker)

- Toma cada archivo recolectado y lo procesa:
 - Clasificación del archivo.
 - Determina si existen parsers capaces de extraer eventos del archivo.
 - Ejecuta filtros si existen.
 - Envía lo extraído a la cola de almacenamiento.

4. Almacenamiento (Storage)

- Escribe todos los eventos de la cola, organiza y graba en un **archivo en formato plaso**.
- El archivo contendrá:
 - cómo y cuándo se ejecutó la herramienta,
 - info del preprocesamiento,
 - metadatos sobre cada contenedor/almacenamiento,
 - parsers utilizados y parámetros,
 - recuento de eventos por parser,
 - tags aplicados y su uso,
 - resumen de plugins de análisis ejecutados y reports asociados.

17 — PLASO: COMPONENTES básicos (herramientas incluidas)

- `image_export` — exportar archivos desde imagen según filtros.
 - `log2timeline` — (legacy name / entrypoint) crea archivo plaso.
 - `pinfo` — información del archivo plaso.
 - `preg` — (herramienta de patrones).
 - `psort` — postproceso y extracción/export del archivo plaso.
-

18 — IMAGE_EXPORT (comando)

- **Uso:** exportar contenido de archivos desde una imagen con filtros (extensiones, path, mactimes).
- **Ejemplo:**

```
image_export.py -x "doc,docx,xls,xlsx,ppt,pptx,pdf" --date_filter "crtim  
e, 2013-10-21, 2013-10-23" -w cdrive_docs_export cdrive.e01
```

19 — LOG2TIMELINE / PLASO (uso básico)

- **Comando para crear archivo de almacenamiento plaso:**

```
$ log2timeline.py --storage-file test.plaso test.vhd
```

- **Contenido del archivo plaso:** eventos extraídos + metadatos sobre el proceso, etiquetas, reports de plugins.

20 — Parsers y plugin presets (info de info)

- Plaso incluye presets de parsers por plataforma (android, linux, macos, etc.) que agrupan parsers relevantes (ej.: `chrome_cache`, `filestat`, `sqlite/*`, `syslog`, `webhist`, etc.).
- **Comando de referencia:**

```
log2timeline.py --info
```

Muestra presets y parsers disponibles.

21 — Uso y ejemplos de log2timeline (colección)

- **Ejemplo de ejecución sobre dispositivo/imagen:**

```
log2timeline.py --storage-file disco.plaso /dev/sda
```

- Plaso solicitará el **identificador de partición** a procesar o `all` para todas.

22 — COLLECTION FILTERS (recolección dirigida / targeted collection)

- **Idea:** cuando se conocen archivos relevantes, usar filtros de recolección para que Plaso solo extraiga eventos de esos archivos (reduce tiempo y ruido).
- **Métodos soportados:**
 - Artefactos forenses (Forensic Artifacts).
 - Files de filtros (Filter files).
- **Ejemplo de uso con filter file:**

```
$ log2timeline.py -f filter.txt --storage-file test.plaso test.vhd
```

Contenido ejemplo (`filter.txt`):

```
{sysregistry}/.+  
/Users/./NTUSER.DAT  
/Documents And Settings/./NTUSER.DAT
```

23 — FORENSIC ARTIFACTS (artefactos forenses)

- **Definición y propósito:**
 - Datos rápidos y comunes que los investigadores necesitan (logs, servicios configurados, cron jobs, estado de parches, cuentas de usuario, etc.).
 - Se almacenan en un repositorio de artefactos reutilizable (mantener rutas, formatos por SO).
 - **Analogía arqueológica:**
 - Artefacto = objeto de interés arqueológico digital.
 - **Ejemplo (formato de artefacto):** definición del artefacto `ApplicationEventLog` con `collector_type: FILE` y paths; otro ejemplo es `ChromeHistory` con paths específicos por SO.
 - **Repositorios:** existe un repositorio mantenido (proyecto Google) con artefactos forenses reutilizables.
-

24 — Ejemplo de artefacto: ChromeHistory

- **Definición (fragmento):**
 - `name: ChromeHistory`
 - `doc: Chrome browser history.`
 - `sources: - type: FILE`
 - `attributes.paths:`
 - rutas tipo `'%%users.localappdata%%\Google\Chrome\User Data\*\History'` y variantes
 - `supported os: [Windows]`
 - **Uso:** permite a Plaso recolectar solo archivos relevantes al historial de Chrome en Windows.
-

25 — log2timeline: colección por artefactos y filtros

- **Comando ejemplo usando artefactos predefinidos:**

```
log2timeline.py --artifact-filters WindowsEventLogSystem --storage-file name.plaso source.vhd
```

- **Salida:** listado de archivos/particiones procesadas y mensajes informativos.

26 — PINFO (información del archivo plaso)

- **Función:** `pinfo` da información sobre meta del archivo plaso (cómo/cuando se ejecutó, parsers usados, conteos de eventos, etc.).
- **Ejemplo:**

```
$ pinfo.py test.plaso
```

Salida (fragmento) muestra `Storage file`, `Serialization format`, `Time of processing`, `os_detected`, `workers`, `method`, `preprocess`, `version`, etc.

27 — PSORT (postprocesado)

- **Qué hace:** postprocesar un archivo de almacenamiento plaso; permite filtrar, ordenar, analizar y exportar a varios formatos (XLSX, JSON, Elastic, timesketch, CSV, etc.).
- **Módulos de salida (ejemplos):**
 - `xlsx`, `elastic`, `4n6time_sqlite`, `dynamic`, `rawpy`, `json`, `json_line`, `timesketch`, etc.
- **Ejemplo de uso para Timesketch:**

```
$ psort.py -o timesketch test.plaso  
$ psort.py -o l2tcsv -w test.l2tcsv test.plaso
```

- **Análisis adicionales:** `psort` tiene plugins de análisis (`browser_search`, `chrome_extension`, `file_hashes`, `tagging`, `viper`, `virustotal`, `windows_services`, etc.).
 - **Tagging:** permite etiquetar eventos que coincidan con reglas (archivo `tag_windows.txt` por defecto).
-

28 — PSORT: FILTROS y Time Slices

- **Time Slices:** filtrar alrededor de una marca de tiempo de interés.
 - Por defecto ± 5 minutos; ajustable con `-slice_size`.
 - Ejemplo:

```
$ psort.py -q --slice "2004-09-20 16:13:02" test.plaso
$ psort.py -q --slice "2004-09-20 16:13:02" --slice_size 100 test.plaso
```

- **Filtros complejos:** `psort` acepta filtros tipo SQL/expresiones sobre campos:
 - Ejemplo:

```
$ psort.py -q test.plaso "date < '2004-09-20 16:20:00' and date > '2004-09-20 16:13:02'"
$ psort.py -q test.plaso "cached_file_size is 43"
$ psort.py --slicer -q test.plaso "cached_file_size is 43"
$ psort.py --slice_size 15 --slicer -q test.plaso "cached_file_size is 43"
```

29 — PSTEAL (combinación práctica)

- **psteal:** combinación de `log2timeline.py` y `psort.py` (facilita flujo).
- **Ejemplos:**

```
psteal.py --source disco.dd -w /tmp/registrar.csv  
psteal.py --source disco.dd -o timesketch
```

30 — TIMESKETCH (visualizador de timelines)

- **Qué es:** herramienta de análisis de líneas de tiempo creada por Google.
 - Usa **Opensearch/Elasticsearch** como base de búsqueda.
 - Ofrece visualización de múltiples timelines, multiusuario, anotaciones, tags, sketches (casos) y timelines dentro de sketches.
- **Formatos de importación:** JSON/JSONL/CSV (formato app), Plaso file (`.plaso`).
- **Uso típico:** crear un Sketch (caso) y subir un `.plaso` o importar con `psort / timesketch-import-client`.

31 — HANDS-ON (instalación y uso con Docker)

- **Plaso con Docker (ejemplo):**

```
docker run -v /tmp:/data log2timeline/plaso log2timeline.py --storage_file /data/disk.plaso /data/source.img
```

- **Timesketch despliegue (docker-compose):**

- Descargar script y ejecutar:

```
curl -s -O https://raw.githubusercontent.com/google/timesketch/master/contrib/deploy_timesketch.sh  
chmod 755 deploy_timesketch.sh  
sudo ~/deploy_timesketch.sh # crea carpeta timesketch  
cd timesketch
```

```
sudo docker-compose up -d
sudo docker-compose exec timesketch-web tsctl create-user <USERNAME>
```

- **Nota:** el `docker-compose` abrirá Timesketch en el puerto 80 de localhost (por defecto).
- **Subida de archivo plaso a Timesketch:** crear un Sketch y subir `disk.plaso`. Puede tardar minutos según tamaño ya que internamente ejecuta `psort` y crea un índice en Elasticsearch/Opensearch.
- **timesketch-import-client (para archivos grandes):**

```
pip3 install timesketch-import-client
timesketch_importer --sketch-id <id> disk.plaso
```

- Durante uso solicita `host_uri`, `auth_mode`, `username` y `password`.
- La configuración se guarda en `~/timesketchrc`.

32 — Problemas / compatibilidad de versiones

- A veces la versión de **plaso** usada por Timesketch puede ser más vieja que la del Docker de Plaso.
- **Solución:** actualizar la versión de plaso dentro del contenedor:

```
docker exec -it timesketch_timesketch-worker_1 bash
root@container:/# apt update; apt upgrade
```

33 — Consideraciones de rendimiento y configuración de Timesketch

- **Memoria de Elasticsearch:** por defecto toma la **mitad de la RAM total**. Se puede reducir mínimo a 2G en `timesketch/config.env` (`ELASTIC_MEM_USE_GB`).
 - **Restart policy:** Timesketch en docker-compose suele tener `restart: always` ; se recomienda cambiar a `restart: unless-stopped` .
 - **Importante:** para archivos `.plaso` muy grandes la interfaz web puede tirar error, por eso el `timesketch_importer` es recomendado.
-

34 — OPENSEARCH (motor de búsqueda / base para Timesketch)

- **Qué es:** base de datos NoSQL y motor analítico distribuido, open-source; indexa datos de todo tipo (textuales, numéricos, geoespaciales).
 - **Características:** API REST potente, rápida indexación y búsquedas gracias a tokenización y normalización.
 - **Índices:** colección de documentos relacionados; los documentos son JSON con campos y valores.
 - **Tokenización:** divide texto en términos, normaliza (minúsculas) y elimina puntuación durante indexación.
 - **Busquedas:** Query DSL; en el texto se menciona referencia a la guía de Query String Query.
-

35 — BÚSQUEDAS en Opensearch / Timesketch

- **Query string:** la búsqueda se divide en términos separados por espacios y operadores (`or` , `and` , `||` , `&&`).
 - *El espacio en blanco cuenta como "or".*
- **Ejemplos de búsqueda:**
 - Buscar palabras sueltas: `"ssh accept"` devolverá eventos que contengan `ssh` o `accept` .
 - Buscar por campo:

```
event.file: /var/log/authlog and event.description: Accept
```

devuelve eventos con `Accept` en descripción y `event.file` igual a `/var/log/authlog`.

- **Filtros:** se pueden filtrar por rango de tiempo, valores de campos, filtros negados (ej.: excluir logs). Las queries se ejecutan sobre eventos filtrados.

36 — PROBLEMAS Y HABILIDADES NECESARIAS (resumen final)

- **Gran problema de las timelines:** aunque los graficadores/visualizadores simplifican la búsqueda, el investigador *debe saber dónde buscar* (conocimiento del sistema operativo, mactimes y patrones comunes).
- **Requisitos para buen análisis de timelines:**
 - Conocimiento profundo del **sistema operativo** analizado (particularidades de timestamps, mactimes).
 - Reconocer **patrones comunes** (actualizaciones del sistema, logins regulares).
 - Saber **filtrar** y **enriquecer** timelines (añadir logs, artefactos) para obtener contexto relevante.
 - Uso de herramientas: `fls`, `mactime`, **Plaso** (log2timeline), `psort`, `pinfo`, **Timesketch**, `image_export`, y repositorio de **Forensic Artifacts**.

37 — Comandos y fragmentos clave (conservados del texto)

Incluyo de forma compacta los ejemplos de comandos y outputs citados para que los tengas como referencia rápida:

- `fls -z GMT -s 0 -m C: -f ntfs -r disco-c.dd > hddisk-c.body`
- Fragmento salida `fls`:

```
0|C:/WINDOWS/inf/mdmpin.PNF|4718-128-3|r/rrwxrwxrwx|0|0|19268|1
299255392|1299255392|1299255392|129925
```

- `mactime -b harddisk-c.body -Z GMT -D > disk-c.csv`

- Fragmento salida `mactime` :

```
Fri Mar 04 2011 16:08:04 618605 .ac. r/rrwxrwxrwx C:/WINDOWS/syste
m32/dllcache/fp4autl.dll 0 0 10618-128-1
```

- `image_export.py -x "doc,docx,xls,xlsx,ppt,pptx,pdf" --date_filter "crttime, 2013-10-21, 2013-10-23" -w cdrive_docs_export cdrive.e01`
- `log2timeline.py --storage-file test.plaso test.vhd`
- `log2timeline.py --artifact-filters WindowsEventLogSystem --storage-file name.plaso test.vhd`
- `pinfo.py test.plaso`
- `psort.py -o timesketch test.plaso`
- `psort.py -o l2tcsv -w test.l2tcsv test.plaso`
- Ejemplos de psort slices/filtrado:

```
$ psort.py -q --slice "2004-09-20 16:13:02" test.plaso
$ psort.py -q test.plaso "cached_file_size is 43"
```

- Plaso con docker (ejemplo):

```
docker run -v /tmp:/data log2timeline/plaso log2timeline.py --storage_file /data/disk.plaso /data/source.img
```

- Timesketch deploy (resumen):

```
curl -s -O https://raw.githubusercontent.com/google/timesketch/master/contrib/deploy_timesketch.sh
chmod 755 deploy_timesketch.sh
```

```
sudo ~/deploy_timesketch.sh
cd timesketch
sudo docker-compose up -d
sudo docker-compose exec timesketch-web tsctl create-user <USERN
AME>
```

- timesketch_importer:

```
pip3 install timesketch-import-client
timesketch_importer --sketch-id <id> disk.plaso
```

38 — Sugerencias prácticas de estudio (para que te sirva a futuro)

- **Repetir el flujo completo** en un entorno controlado (imagen forense pequeña): `fls` → `mactime` y luego `plaso` (log2timeline) → `psort` → Timesketch.
- **Practicar filtros y slices** en `psort` para extraer contextos temporales.
- **Familiarizarte con artefactos** (Chrome, Windows Event Logs, etc.) y cómo están definidos para la recolección dirigida.
- **Probar Plaso con diferentes presets** (linux, macos, android) para ver qué parsers se activan.
- **Configurar Timesketch con recursos adecuados** (ajustar `ELASTIC_MEM_USE_GB`) y usar `timesketch_importer` para grandes archivos.